

Chainlink Functions — Mastery Notes

🔗 Core Problem Chainlink Functions Solves

Question:

How do I let a smart contract trigger a *private API request* without exposing secrets, while keeping execution *verifiable* and *decentralized*?

Why this problem exists

- Smart contracts are **public**
- Frontend JavaScript is **public**
- Secrets **cannot live on-chain**
- Anyone can **replay calls**
- Centralized servers break decentralization

🔗 **Chainlink Functions exists to solve exactly this problem.**

🌟 Recommended Learning Path

```
5-use-secrets-threshold
→ 9-send-cbor
→ 8-multiple-apis
→ 6-use-secrets-gist
→ 7-use-secrets-url
```

🔒 use-secrets-threshold (MAX SECURITY)

What it is

- Secrets are **sharded**
- Stored across **multiple DON nodes**
- **No single node** knows the full secret

What you gain

- ☒ Highest security
- ☒ Enterprise-grade
- ☒ Exchange-grade key safety

What you lose

- ☒ Setup complexity
- ☒ Harder debugging

When to use

- Custodial funds
- Exchange API keys (Binance, Aster, OKX)
- Anything that can move money

Mental model

"This is like splitting a private key across 10 HSMs."



use-secrets-gist

What it is

- Secrets stored in a **private GitHub Gist**
- Chainlink DON fetches secrets securely

What you gain

- ☒ Very easy setup
- ☒ No local encryption tools
- ☒ Perfect for demos / PoCs

What you lose

- ☒ GitHub trust assumption
- ☒ Not ideal for production funds

When to use

- Hackathons
- Early-stage products
- Read-only APIs

Mental model

"Secrets live in a locked vault on GitHub instead of your laptop."



use-secrets-url

What it is

- Secrets fetched from **any HTTPS endpoint**
- Can integrate:
 - AWS Secrets Manager
 - GCP Secret Manager
 - HashiCorp Vault
 - Custom backend

What you gain

- ☒ Full control
- ☒ Key rotation
- ☒ Enterprise infra compatible

What you lose

- ✗ You must secure the endpoint

When to use

- Real products
- Compliance-heavy systems
- Exchange integrations

Mental model

“Chainlink DON asks your server for secrets, never your contract.”

send-cbor (ADVANCED / PROTOCOL LEVEL)

What it is

- Manually encode the **CBOR request**
- Full control over:
 - arguments
 - secrets
 - callbacks
 - gas usage

Why this exists

- `sendRequest()` is a **convenience wrapper**
- `sendCBOR()` gives **raw protocol access**

What it unlocks

- Batch requests
- Reusable payloads
- Meta-oracles
- Forwarded / relayed requests

When to use

- Advanced workflows
- Multi-hop oracles
- Relayers
- Protocol-level tooling

Mental model

"sendRequest is REST, sendCBOR is raw TCP."

Final Takeaway

Chainlink Functions is **not about fetching data**.

It is about:

- Secure computation
- Private secrets
- Verifiable execution
- Decentralized trust

Once you master:

- `use-secrets-threshold`
- `send-cbor`

You can build:

- On-chain trading engines
 - Exchange-integrated protocols
 - Institutional-grade DeFi infrastructure
-

 **This is oracle engineering, not API calling.**